

## Problem A. Play with Binary Numbers

|                          |            |
|--------------------------|------------|
| <b>Time Limit</b>        | 1201 ms    |
| <b>Mem Limit</b>         | 1572864 kB |
| <b>Code Length Limit</b> | 50000 B    |
| <b>OS</b>                | Linux      |

Let  $S$  be the binary representation of an integer. We define two functions  $a(i)$  and  $b(i)$  such that:

- $a(i)$  = Number of occurrences of '1' at odd positions of  $S$ ,
- $b(i)$  = Number of occurrences of '1' at even positions of  $S$ .

For example: for integer 19,  $S = 10011$ , so  $a(19) = 2$  and  $b(19) = 1$ .

### Input

First line contains an integer  $T$ , the number of test cases. Then  $T$  lines follow. On each line, you will be given three integers  $M$ ,  $N$ ,  $K$ .

### Output

For each test case output a single integer  $R$ , where  $R$  is the number of integers 'i' between  $M$  and  $N$  (both inclusive) such that absolute difference of  $a(i)$  and  $b(i)$  is equal to  $K$ . The answer to each test case should be on a separate line.

### Constraints

$$T \leq 50$$

$$1 \leq M < N \leq 10^{19}$$

$$1 \leq N - M \leq 10^6$$

$$0 \leq K \leq 50$$

### Example

| Input       | Output |
|-------------|--------|
| 1<br>1 10 2 | 2      |

## Problem B. Fedor and New Game

**Time Limit** 1000 ms

**Mem Limit** 262144 kB

**Input File** stdin

**Output File** stdout

After you had helped George and Alex to move in the dorm, they went to help their friend Fedor play a new computer game «Call of Soldiers 3».

The game has  $(m + 1)$  players and  $n$  types of soldiers in total. Players «Call of Soldiers 3» are numbered from 1 to  $(m + 1)$ . Types of soldiers are numbered from 0 to  $n - 1$ . Each player has an army. Army of the  $i$ -th player can be described by non-negative integer  $x_i$ . Consider binary representation of  $x_i$ : if the  $j$ -th bit of number  $x_i$  equal to one, then the army of the  $i$ -th player has soldiers of the  $j$ -th type.

Fedor is the  $(m + 1)$ -th player of the game. He assume that two players can become friends if their armies differ in at most  $k$  types of soldiers (in other words, binary representations of the corresponding numbers differ in at most  $k$  bits). Help Fedor and count how many players can become his friends.

### Input

The first line contains three integers  $n, m, k$  ( $1 \leq k \leq n \leq 20$ ;  $1 \leq m \leq 1000$ ).

The  $i$ -th of the next  $(m + 1)$  lines contains a single integer  $x_i$  ( $1 \leq x_i \leq 2^n - 1$ ), that describes the  $i$ -th player's army. We remind you that Fedor is the  $(m + 1)$ -th player.

### Output

Print a single integer — the number of Fedor's potential friends.

### Examples

| Input                        | Output |
|------------------------------|--------|
| 7 3 1<br>8<br>5<br>111<br>17 | 0      |

| Input                     | Output |
|---------------------------|--------|
| 3 3 3<br>1<br>2<br>3<br>4 | 3      |

## Problem C. Raising Bacteria

**Time Limit** 1000 ms

**Mem Limit** 262144 kB

You are a lover of bacteria. You want to raise some bacteria in a box.

Initially, the box is empty. Each morning, you can put any number of bacteria into the box. And each night, every bacterium in the box will split into two bacteria. You hope to see exactly  $X$  bacteria in the box at some moment.

What is the minimum number of bacteria you need to put into the box across those days?

### Input

The only line containing one integer  $x$  ( $1 \leq x \leq 10^9$ ).

### Output

The only line containing one integer: the answer.

### Examples

| Input | Output |
|-------|--------|
| 5     | 2      |

| Input | Output |
|-------|--------|
| 8     | 1      |

### Note

For the first sample, we can add one bacterium in the box in the first day morning and at the third morning there will be 4 bacteria in the box. Now we put one more resulting 5 in the box. We added 2 bacteria in the process so the answer is 2.

For the second sample, we can put one in the first morning and in the 4-th morning there will be 8 in the box. So the answer is 1.

## Problem D. Number Place

**Time Limit** 2000 ms

### Problem Statement

There is a  $9 \times 9$  grid  $A$ , where each cell contains an integer between 1 and 9, inclusive. Specifically, the cell at the  $i$ -th row from the top and  $j$ -th column from the left contains  $A_{i,j}$ .

If  $A$  satisfies all of the following conditions, print **Yes**. Otherwise, print **No**.

- For each row of  $A$ , the nine cells in that row contain each integer from 1 to 9 exactly once.
- For each column of  $A$ , the nine cells in that column contain each integer from 1 to 9 exactly once.
- Divide the rows of  $A$  into three groups, each of three rows, from top to bottom, and similarly divide the columns into three groups, each of three columns, from left to right. Each  $3 \times 3$  grid obtained from  $A$  in this way contains each integer from 1 to 9 exactly once.

### Constraints

- $1 \leq A_{i,j} \leq 9$
- All input values are integers.

### Input

The input is given from Standard Input in the following format:

```
A1,1 A1,2 ... A1,9
A2,1 A2,2 ... A2,9
⋮
A9,1 A9,2 ... A9,9
```

### Output

If the grid  $A$  satisfies all the conditions in the problem statement, print **Yes**; otherwise, print **No**.

### Sample 1

| Input                                                                                                                                                                          | Output |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre> 1 2 3 4 5 6 7 8 9 4 5 6 7 8 9 1 2 3 7 8 9 1 2 3 4 5 6 2 3 4 5 6 7 8 9 1 5 6 7 8 9 1 2 3 4 8 9 1 2 3 4 5 6 7 3 4 5 6 7 8 9 1 2 6 7 8 9 1 2 3 4 5 9 1 2 3 4 5 6 7 8 </pre> | Yes    |

The grid  $A$  is shown below.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 |
| 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 |
| 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 |
| 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

The grid  $A$  satisfies all three conditions, so print **Yes**.

### Sample 2

| Input                                                                                                                                                                                     | Output |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1 2 3 4 5 6 7 8 9<br>2 3 4 5 6 7 8 9 1<br>3 4 5 6 7 8 9 1 2<br>4 5 6 7 8 9 1 2 3<br>5 6 7 8 9 1 2 3 4<br>6 7 8 9 1 2 3 4 5<br>7 8 9 1 2 3 4 5 6<br>8 9 1 2 3 4 5 6 7<br>9 1 2 3 4 5 6 7 8 | No     |

The grid  $A$  is shown below.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 |
| 6 | 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 9 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

For example, if you look at the top left  $3 \times 3$  grid, you can see that the third condition is unsatisfied, so print **No**.

### Sample 3

| Input                                                                                                                                                                                     | Output |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1 2 3 4 5 6 7 8 9<br>4 5 6 7 8 9 1 2 3<br>7 8 9 1 2 3 4 5 6<br>1 2 3 4 5 6 7 8 9<br>4 5 6 7 8 9 1 2 3<br>7 8 9 1 2 3 4 5 6<br>1 2 3 4 5 6 7 8 9<br>4 5 6 7 8 9 1 2 3<br>7 8 9 1 2 3 4 5 6 | No     |



The grid  $A$  is shown below.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |

For example, if you look at the leftmost column, you can see that the second condition is unsatisfied, so print **No**.

## Problem E. Special Fibonacci

**Time Limit** 1000 ms

**Code Length Limit** 50000 B

**OS** Linux

Sankalp recently learned **Fibonacci numbers** and now he is studying different algorithms to find them. After getting bored of reading them, he came with his own new type of numbers. He defined them as follows:

- $f(0) = a$ ;
- $f(1) = b$ ;
- $f(n) = f(n-1) \wedge f(n-2)$ ; when  $n > 1$ , where  $\wedge$  denotes the [bitwise xor operation](#).

You are given three integers  $a, b$  and  $n$ , calculate  $f(n)$ .

### Input

The input contains one or more independent test cases.

The first line of input contains a single integer  $T$  ( $1 \leq T \leq 10^3$ ), the number of test cases.

Each of the  $T$  following lines contains three space-separated integers  $a, b$ , and  $n$  ( $0 \leq a, b, n \leq 10^9$ ) respectively.

### Output

For each test case, output  $f(n)$ .

## Constraints

- $(1 \leq T \leq 1000)$
- $(0 \leq a, b, n \leq 10^9)$

### Sample Input

```
4
86 77 15
93 35 86
92 49 21
62 27 90
```

### Sample Input

86

126

92

62

## Problem F. Coloring Colorfully

**Time Limit** 2000 ms

### Problem Statement

$N$  tiles are arranged in a row from left to right. The initial color of each tile is represented by a string  $S$  of length  $N$ .

The  $i$ -th tile from the left is painted black if the  $i$ -th character of  $S$  is **0**, and painted white if that character is **1**.

You want to repaint some of the tiles black or white, so that any two adjacent tiles have different colors.

At least how many tiles need to be repainted to satisfy the condition?

### Constraints

- $1 \leq |S| \leq 10^5$
- $S_i$  is **0** or **1**.

### Input

Input is given from Standard Input in the following format:

$S$

### Output

Print the minimum number of tiles that need to be repainted to satisfy the condition.

### Sample 1

| Input | Output |
|-------|--------|
| 000   | 1      |

The condition can be satisfied by repainting the middle tile white.

**Sample 2**

| Input    | Output |
|----------|--------|
| 10010010 | 3      |

**Sample 3**

| Input | Output |
|-------|--------|
| 0     | 0      |

## Problem G. Lower

**Time Limit** 2000 ms

### Problem Statement

There are  $N$  squares arranged in a row from left to right.

The height of the  $i$ -th square from the left is  $H_i$ .

You will land on a square of your choice, then repeat moving to the adjacent square **on the right** as long as the height of the next square is not greater than that of the current square.

Find the maximum number of times you can move.

### Constraints

- All values in input are integers.
- $1 \leq N \leq 10^5$
- $1 \leq H_i \leq 10^9$

### Input

Input is given from Standard Input in the following format:

```
N
H1 H2 ... HN
```

### Output

Print the maximum number of times you can move.

### Sample 1

| Input           | Output |
|-----------------|--------|
| 5<br>10 4 8 7 3 | 2      |

By landing on the third square from the left, you can move to the right twice.

### Sample 2

| Input              | Output |
|--------------------|--------|
| 7<br>4 4 5 6 6 5 5 | 3      |

By landing on the fourth square from the left, you can move to the right three times.

### Sample 3

| Input        | Output |
|--------------|--------|
| 4<br>1 2 3 4 | 0      |

## Problem H. Flipping bits

OS Linux

You will be given a list of 32 bit unsigned integers. Flip all the bits ( $1 \rightarrow 0$  and  $0 \rightarrow 1$ ) and return the result as an unsigned integer.

### Example

$$n = 9_{10}$$

$9_{10} = 1001_2$ . We're working with 32 bits, so:

[illegible]

$$1111111111111111111111111110110_2 = 4294967286_{10}$$

Return **4294967286**.

### Function Description

Complete the *flippingBits* function in the editor below.

flippingBits has the following parameter(s):

- *int n*: an integer

## Returns

- *int*: the unsigned decimal integer result

### Input Format

The first line of the input contains  $q$ , the number of queries.

Each of the next  $q$  lines contain an integer,  $n$ , to process.

## Constraints

$$1 \leq q \leq 100$$

$$0 \leq n < 2^{32}$$

| Input      | Output     |
|------------|------------|
| 3          | 2147483648 |
| 2147483647 | 4294967294 |
| 1          | 4294967295 |
| 0          |            |

### Explanation 0



$$01111111111111111111111111111111_2 = 2147483647_{10}$$

$$10000000000000000000000000000000_2 = 2147483648_{10}$$

$$00000000000000000000000000000001_2 = 1_{10}$$

$$11111111111111111111111111111110_2 = 4294967294_{10}$$

$$00000000000000000000000000000000_2 = 0_{10}$$

$$11111111111111111111111111111111_2 = 4294967295_{10}$$

| Input  | Output     |
|--------|------------|
| 2      | 4294967291 |
| 4      | 4294843839 |
| 123456 |            |

### Explanation 1

$$0000000000000000000000000000000100_2 = 4_{10}$$

$$111111111111111111111111111111011_2 = 4294967291_{10}$$

$$000000000000000011110001001000000_2 = 123456_{10}$$

$$111111111111111100001110110111111_2 = 4294843839_{10}$$

| Input     | Output     |
|-----------|------------|
| 3         | 4294967295 |
| 0         | 3492223820 |
| 802743475 | 4259365872 |
| 35601423  |            |

### Explanation 2

$$00000000000000000000000000000000_2 = 0_{10}$$

$$11111111111111111111111111111111_2 = 4294967295_{10}$$

$$00101111110110001110010010110011_2 = 802743475_{10}$$

$$11010000001001110001101101001100_2 = 3492223820_{10}$$

$$00000010000111110011110000001111_2 = 35601423_{10}$$

$$11111101111000001100001111110000_2 = 4259365872_{10}$$